

Project: Dual core machine learning accelerator for attention mechanism

Background: [ECE260B_Project.pdf](#) ↓

Step1. Single core RTL, Synthesis & PNR.

- Single core matrix multiplier RTL and TB is provided in "public/course_contents/ece260_project/" directory.

- *Note: currently the output of core is not connected yet. So, you need to connect based on your design, and then need to run synthesis. Otherwise, you will get zero power and zero delay.*

- *Note: above code is just a template backbone. You are supposed to build up based on the code.*

- You are allowed to change any part of verilog code, like SRAM size, FIFO depth, and others.

- You are supposed to compute $[Q0, Q1, \dots, Q7]^T * [K0, K1, \dots, K7]$, where Q_i and K_i are vectors with 8 elements. Each element is 8bit integer.

- The computation for the following should be processed

e.g., $[Q0K0, Q0K1, \dots, Q0K7]$ @ cyc0, ...

$[Q1K0, Q1K1, \dots, Q1K7]$ @ cyc1, ...

$[Q2K0, Q2K1, \dots, Q2K7]$ @ cyc1, ...

- Synthesize and run PnR to get the WNS, power, and area report by targeting 1GHz

- It is okay not to meet the 0 WNS.

- Input ports are from west, whereas output ports are at south

- You can use qdata.txt and kdata.txt for this sim.

- **Deliverable:** RTL (verilog), pnr-completed .enc file, waveform vcd and snapshot of it from **gate-level sim** to show the functional correctness in report.

How to show the correctness of gate-level sim ? Some internal nodes are hard to find from the gate-level netlist. Simple solution is, store your results in pmem. Then, fetch the result from pmem to the output port of core. The result fetched from output can be compared with your estimated result calculated in the TB such as HW1. (not supposed to compare with norm.txt. The norm.txt is a file to be used as an input for the calculation of subsequent stages.)

Step2. Output normalization

- Your outputs are stored in the partial sum memory (pmem)
e.g., [Q0K0, Q0K1, ..., Q0K7] @ add0,
[Q1K0, Q1K1, ..., Q1K7] @ add1,
[Q2K0, Q2K1, ..., Q2K7] @ add2, ...
- Fetch one by one and compute the summation to normalize it, then write back to the memory
e.g., [abs(Q0K0), abs(Q0K1), ..., abs(Q0K7)] / sum(abs(Q0K0), abs(Q0K1), ..., abs(Q0K7))
[abs(Q1K0), abs(Q1K1), ..., abs(Q1K7)] / sum(abs(Q1K0), abs(Q1K1), ..., abs(Q1K7)) ...
- Any method is fine, e.g., fifo, register, ...
- You can compare your result from hardware with your estimated result.
- **Deliverable:** RTL (verilog), waveform vcd, snapshot of waveform from behavioral sim to show the functional correctness in report

Step3. Hierarchical Synthesis of core.

- In this step, you will have to perform a hierarchical synthesis followed by PnR for the single core you designed in step 1. In this step, the SRAMs will be synthesized and PnR will be performed separately. This will then be used for the synthesis/PnR of the core as a STD cell.
- First synthesize the different SRAMs used in your core with the following specifications:
 - Top metal layer would be M4 (-specifyTopLayer 4).
 - Use a pitch of 4 um for the pins.
 - You can use the following pin placement: "D" input on the bottom edge, "Q" output on the top edge (D[0] & Q[0] start from the left) and the rest of the pins are on the left edge.
 - You have freedom for the other design parameters, e.g., side of macro, pin's alignment from center or start and so on.
- Follow the subsequent steps for Synthesis and PnR by studying the lecture slides and demonstration video from the class.
- You can optimize the placement of the SRAM within the die as well as the pin placement as per your design.
- Hint: The following two functions can be used to rotate or flip any of the hierarchical blocks during PnR:

```
flipOrRotateObject -rotate R90 -name qmem_instance  
flipOrRotateObject -flip MY -name qmem_instance
```
- **Deliverable:** RTL (verilog), pnr-completed .enc file, waveform vcd and snapshot of it from gate-level sim to show the functional correctness in report.

Step4. Hierarchical Synthesis of dual core.

- Ideally, you want to process 16 vectors,

e.g., [Q0K0, Q0K1, ..., Q0K15] @ cyc0, ...

[Q1K0, Q1K1, ..., Q1K15] @ cyc1, ...

But, your core only supports up to 8 vectors.

- Process in parallel with dual cores

- But, the sum of vector elements should be merged for normalization,

e.g., $\text{sum}(\text{abs}(\text{Q0K0}), \text{abs}(\text{Q0K1}), \dots, \text{abs}(\text{Q0K7}))$ from core0

+ $\text{sum}(\text{abs}(\text{Q0K8}), \text{abs}(\text{Q0K9}), \dots, \text{abs}(\text{Q0K15}))$ from core1

- Two cores need to exchange their sums, but note the core0 and core1 operate on separate CLK domains.

- Any asynchronous protocol can be used if it is technically correct.

- The asynchronous protocol can be part of the core or separate from it.

- The dual core design must be synthesized hierarchically from two single cores following a similar standard procedure as step 3.

- Deliverable: RTL (verilog), pnr-completed .enc file, waveform vcd and snapshot of it from gate-level sim to show the functional correctness in report

For functional correctness, what should be shown ? Validate your normalization output by comparing the estimated results calculated from the tb (use kdata_core0/1.txt and qdata.txt as input. qdata.txt is common for both cores). For the "norm * value" calculation, the output can be validated by comparing the estimated results calculated from the tb (use vdata.txt and norm_core0/1.txt as input. vdata.txt is common for both cores). Note "norm * value" computation does not require any new hardware. It is nearly the same as Q * K, but you need to apply the data properly as shown in FAQ1.

Step5. Optimization & PNR

- [TBD: This section will be updated]

- **Note:** *This project is not spec competition. Your logical approach and idea are more important*

- **Deliverable:** *RTL (verilog), pnr-completed .enc file, waveform vcd and snapshot of it from gate-level sim to show the functional correctness in report*

Report :

- Only one of the group member is supposed to submit the report.

- Include all the deliverable's path in a table (only one of the group member needs to have the files).

- Prove the efficacy of your idea with clear comparison of before vs. after to show how much performance/improvement achieved

- Measure power and check functionality with VCD

(SDF is not required, but it will be plus alpha)

- Clear description in the report regarding your design choice

e.g., Why you chose the way of implementation, Why you cut the pipeline boundary,etc

- Your report should be no more than 4 pages (fewer is fine; not required to fill) with roughly these sections:

- Introduction (roughly introduce what you're doing; can be simply one paragraph)

- Sections for all steps + a performance analysis section (or subsection)

- Conclusion

- You can also add other sections you think necessary.

- The snapshot of waveform to show the functional correctness needs to be attached for the last step. (If you didn't complete all the steps, provide the last step's snapshot)

- TBD

1. what is our output of $\text{norm}(Q \cdot K)$ stage, and $\text{norm} \cdot V$ stage ?

cycle1: [norm(Q1*K1), norm (Q1*K2),norm(Q1*KN)]

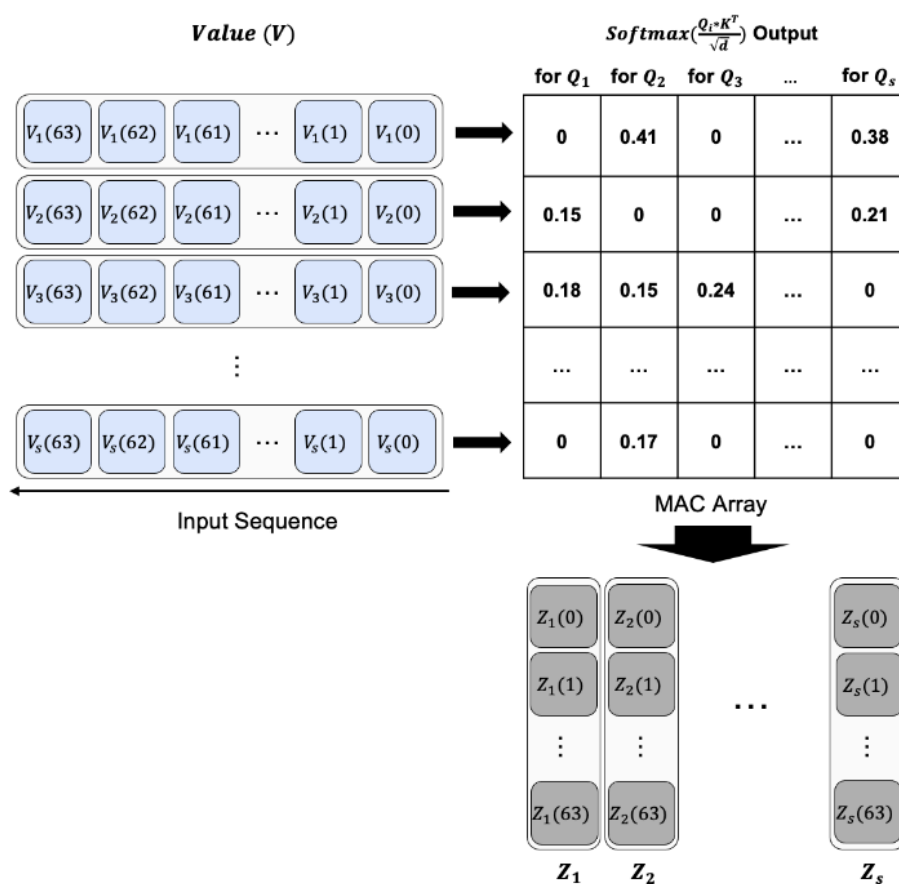
cycle2: [norm(Q2*K1), norm (Q2*K2),norm(Q2*KN)]

...

See the below conceptual figure.

You may simply store the `ndata.txt` in the array. Then, send the first element of 1st row, 2nd row ... 8th row in the `vdata.txt` at `cycle1`, and 2nd elements of them at `cycle2`, and so on.

Note below figure is assuming 64 element per vector. But, in our hardware, it should be just 8.



2. I have many hi-Z values after gate-level sim. (See FAQ3 as well for further discussion)

In the run_gui file, please add following options below the xrun command.

```
-define INITIALIZE_MEMORY \  
-xminitialize 0 \
```

3. I have used the options of "-ncinitialize 0" and "-define INITIALIZE_MEMORY". But, I still have "XX" states.

Please check whether you forgot to add reset for an important signal, or connection is wrong.

If not, please try "-ncinitialize rand_2state: 1". Here 1 is a seed number and you are supposed to initialize all the nodes with a random set of binary values by using this command. Then, try with a couple of different seeds.

If above does not work, please try to narrow down which node is the source of X-state. Then, you may try "deposit" command. <https://stackoverflow.com/questions/22959183/how-to-use-verilog-deposit-with-indexes>

As one of the extreme experiments, you could copy the "tcbn65gplus.v" into your own directory. Then, at the end of the file, there is a part to define "tsmc_dff"'s truth table. There, you can replace all the "X" into a certain binary value so that you can prohibit the generation of "X" from register. But, this method could cause some malfunction in a certain case. So, not very recommended.

Even if you do not apply any of above methods, it could work suddenly once you simply re-synthesize. Or, even if your synthesized version does not work, your PnRed version could work.

4. Do I need to check empty and full condition ?

It is a plus, but it is fine not to equip by assuming your instruction provider (programmer) will be able to manage to avoid those conditions.

5. For single core or dual core version (before optimization), do we need to meet timing ?

No, as you haven't applied any pipeline, multi-cycle and others, you won't be able to meet 1GHz in Step1 or dual core before optimization.

Please try with relaxed clock frequency initially. Then, you can show the improvement by meeting 1GHz after all optimizations.

6. Are we supposed to use the output of norm(Q*K) stage as an input of norm*v stage ?

It is good, but not required. You may use just provided norm.txt file.

7. The output of norm(Q*K) stage should be identical to the provided norm.txt file ?

No, it is not.

8. Do I need to use "FIFO" style for async interface ?

Not, necessarily. FIFO might be faster in terms of data transaction bandwidth, but other options are also fine.

9. My synthesis / PnR takes so long.

Then, you might check your sdc file. You might be building the hardware with impossible timing condition.

10. Do I need to care the timing of reset path ?

No, you may use "set_false_path" command for reset.

11. In the normalization process, both nominator and denominators should be absoluted ?

Yes, both of them. In the cartoon in the project explanation slide, the abs is missing in the nominator mistakenly.

But, as described in the official project description in this page, you should apply absolute for both.

12. How to verify Normalized output?

You need to create stimulus from TB end using qdata, kdata, then, hook the norm out from design and verify/print the outputs to check by building a simple test function in the TB.

13. The final version should included all the features listed, e.g., dual core, async interface and others ?

Yes, your final version should include everything.

14. Our team members are dropping the course. How about the policy ?

As given in the lecture 1, a single member team has 10% benefit, and 2-member team has 5% benefit in score.

15. Am I supposed to modify TB ?

Yes, you should.

16. SFP should be in the middle between mac array and ofifo ?

No, SFP should be attached after OFIFO.

17. Is there any port limitation ?

No, it is completely free to change the ports of any block.

18. Which version am I supposed to show the synthesis and PNR results ?

You should submit the synthesis and pnr results for 1. single core version and 2. final dual core optimized version.

But, you might want to do PNR for the dual core of un-optimized version to show the improvement.

19. Are we supposed to meet the timing for Worst corner ?

Such as other homeworks, you do not need to meet the Worst corner, but should meet at TYP corner.

However, you should include Worst or Best corners during the synthesis and PnR.

21. Am I supposed to use the only 1 tb file ?

No, you can have multiple separate tb files for each test.

22. My SFP should receive the input from ofifo directly or pmem ?

In general, pmem's output is fed as an input of SFP, but you may directly connect the ofifo's output as an input of SFP with proper modifications.

23. If I cannot verify the functionality through gate-level sim, my score will be deducted ?

Yes

24. Do we need to use SDF file for gate-level sim ?

It is not required but good to have.

25. The very final computed results should be stored back in the pmem ?

Yes, it is recommended. So, your pmem output could be connected to the output port. Then, your final results fetched from pmem can be validated in the test bench.

26. There is a "sfp_row.v" file in the verilog directory. Are we supposed to use this ?

Not necessarily. It is just an example. You could exploit by modifying it or build your own version.

27. Sub-modul has many LVS error of dangling nodes.

It is expected as you do not have a power ring. It is okay.

28. Why we have sub-module LVS violations ?

Since you did not setup a power ring for submodule pnr, it is expected to have a dangling node. As we don't have a ring you will get these violations.

29. The provided code has 16 input MAC. Should I change to be 8 input ?

Yes, the provided code is just an example.